

MÓDULO 1. FUNDAMENTOS AVANZADOS DE JAVASCRIPT

1.1 Introducción a JavaScript (Enfoque moderno y profesional)

1.1.1 Naturaleza del lenguaje y evolución

JavaScript es un lenguaje de programación interpretado, dinámico y multiparadigma, cuyo núcleo se ejecuta en motores como **V8 (Chrome, Node.js)**, **SpiderMonkey (Firefox)** o **JavaScriptCore (Safari)**. Aunque tradicionalmente se concibió como un lenguaje de scripting para navegadores, hoy en día forma parte de un ecosistema completo que abarca:

- Desarrollo frontend (SPAs)
- Backend (Node.js)
- Aplicaciones móviles híbridas (Ionic + Capacitor)
- Aplicaciones de escritorio (Electron)
- Microservicios y serverless

Desde ES5 (2009) hasta ES6 (2015) y versiones posteriores (ES2016+), JavaScript ha evolucionado significativamente, incorporando características que permiten escribir código más estructurado, mantenible y cercano a paradigmas más formales.

Uno de los aspectos clave del JavaScript moderno es la transición desde un estilo imperativo hacia un enfoque más declarativo y modular.

1.1.2 Entorno de ejecución y flujo de ejecución

JavaScript se ejecuta en un entorno controlado por un motor que implementa:

- Compilación Just-In-Time (JIT)
- Gestión de memoria automática (Garbage Collector)
- Loop de eventos (Event Loop)

El **Event Loop** es una de las piezas fundamentales para comprender cómo funciona JavaScript, especialmente en contextos asíncronos.

Ejemplo conceptual del Event Loop

```
console.log("Inicio");
```

```
setTimeout(() => {  
    console.log("Callback asíncrono");  
}, 0);
```

```
console.log("Fin");
```

Resultado esperado:

```
Inicio  
Fin  
Callback asíncrono
```

Esto ocurre porque JavaScript no ejecuta directamente la función del `setTimeout`, sino que la delega en la Web API y la coloca en la cola de tareas (callback queue), siendo ejecutada solo cuando la pila de ejecución (call stack) está vacía.

Este modelo es fundamental para entender frameworks como Ionic o Angular, donde las operaciones asíncronas son constantes.

1.1.3 Configuración del entorno de desarrollo (VS Code)

Para un entorno profesional de desarrollo, utilizaremos **Visual Studio Code** con una configuración orientada a calidad de código y productividad.

Extensiones recomendadas

- ESLint → análisis estático
- Prettier → formateo de código
- JavaScript (ES6) code snippets
- Debugger for Chrome
- Angular Language Service (si se usa Ionic con Angular)

Configuración básica (settings.json)

```
{
  "editor.formatOnSave": true,
  "editor.tabSize": 2,
  "eslint.validate": ["javascript"],
  "files.autoSave": "onFocusChange"
}
```

Ejecución de código

Para trabajar sin backend inicialmente:

Opción 1: Node.js

node archivo.js

Opción 2: Navegador

- Crear archivo HTML
- Incluir script JS

```
<!DOCTYPE html>
<html>
<head>
  <title>Test</title>
</head>
<body>
  <script src="app.js"></script>
</body>
</html>
```

1.1.4 JavaScript como lenguaje dinámico y débilmente tipado

JavaScript no requiere declaración explícita de tipos, lo que introduce flexibilidad pero también posibles errores en tiempo de ejecución.

Ejemplo

```
let dato = 10;  
dato = "texto"; // válido
```

```
console.log(typeof
```

Esto implica que el desarrollador debe aplicar buenas prácticas para evitar comportamientos inesperados.

Caso práctico: error típico

```
function sumar(a, b) {  
  return a + b;  
}
```

```
console.log(sumar(10, "5")); // "105"
```

Solución:

```
function sumar(a, b) {  
  return Number(a) + Number(b);  
}
```

1.1.5 Scope, contexto y hoisting (visión inicial)

Uno de los conceptos más importantes en JavaScript avanzado es el manejo del ámbito de las variables (scope) y el contexto (**this**).

Hoisting

JavaScript eleva las declaraciones de variables y funciones al inicio de su contexto.

```
console.log(a); // undefined  
var a = 5;
```

Equivalente real:

```
var a;  
console.log(a);  
a = 5;
```

Con **let** y **const**, esto cambia debido al **Temporal Dead Zone (TDZ)**:

```
console.log(a); // Error  
let a = 5;
```

1.1.6 Paradigmas soportados

JavaScript permite múltiples paradigmas:

Imperativo

```
let suma = 0;
for (let i = 0; i < 10; i++) {
  suma += i;
}
```

Funcional

```
const suma = [1,2,3,4].reduce((acc, val) => acc + val, 0);
```

Orientado a objetos (basado en prototipos)

```
function Persona(nombre) {
  this.nombre = nombre;
}
```

```
Persona.prototype.saludar = function() {
  return Hola, soy ${this.nombre};
};
```

```
const p = new Persona("Ana");
console.log(p.saludar());
```

Versión moderna:

```
class Persona {
  constructor(nombre) {
    this.nombre = nombre;
  }

  saludar() {
    return Hola, soy ${this.nombre};
  }
}
```

1.1.7 Buenas prácticas desde el inicio

Aunque muchos conceptos se desarrollarán más adelante, desde el primer momento es importante aplicar:

- Uso de `const` por defecto
- Uso de `let` en lugar de `var`
- Evitar variables globales
- Modularización del código
- Nombres claros y semánticos

Ejemplo comparativo

Código poco profesional:

```
var x = 5;  
var y = 10;  
var z = x + y;
```

Código profesional:

```
const precioBase = 5;  
const impuesto = 10;  
const total = precioBase + impuesto;
```

1.1.8 Caso práctico guiado (VS Code)

Objetivo:

Simular un pequeño sistema de cálculo de precios.

```
// app.js

function calcularPrecio(base, impuesto) {
  if (typeof base !== "number" || typeof impuesto !== "number") {
    throw new Error("Los valores deben ser numéricos");
  }

  return base + (base * impuesto / 100);
}

try {
  const resultado = calcularPrecio(100, 21);
  console.log(Precio final: ${resultado});
} catch (error) {
  console.error(error.message);
}
```

Conceptos implicados:

- Validación de tipos
- Manejo de errores
- Funciones puras
- Separación lógica

1.1.9 Relación con Ionic

Es importante que el alumnado entienda desde el principio que:

- Toda la lógica de Ionic se basa en JavaScript/TypeScript
- El manejo del estado y eventos depende del modelo JS
- El consumo de APIs se basa en asincronía

Por tanto, dominar JavaScript moderno es un requisito imprescindible antes de construir aplicaciones móviles.

Conclusión

La introducción a JavaScript en un contexto avanzado no consiste en aprender sintaxis básica, sino en:

- Comprender el modelo de ejecución
- Adoptar estándares modernos (ES6+)
- Desarrollar buenas prácticas desde el inicio
- Entender cómo el lenguaje soporta aplicaciones complejas